

STRIPE PROJECT

2. OLAP Data System

D3. OLAP System Schema Design

Nicolas Pichon - AIA RNCP 38777 / BC02 / D3 : OLAP System Schema Design / v08 - 2026/10/13.

D3.1. Contexte

Le présent document présente le schéma physique de l'entrepôt analytique : type de schéma, organisation en étoile par processus métier, stratégie d'agrégation, et techniques d'optimisation des requêtes.

Notation crow's foot (| | un et un seul, o{ zéro à plusieurs).

D3.2. Schémas en étoile

Dans le schéma en étoile (*Star*), chaque dimension est directement reliée à la table de faits, sans hiérarchie de sous-dimensions normalisées.

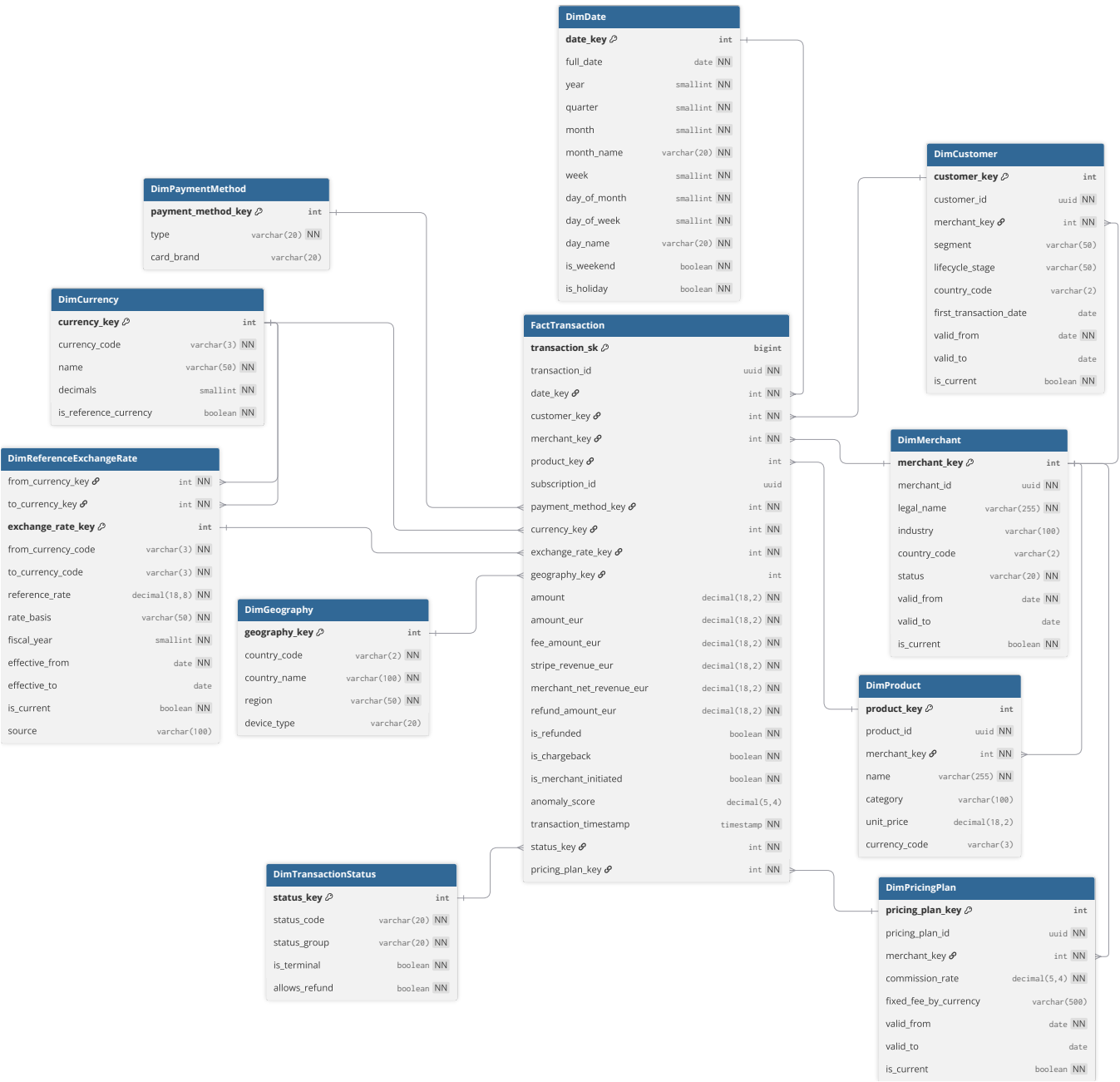
Justification

Le flocon (snowflake) réduit la redondance au prix de jointures supplémentaires ; en lecture analytique, c'est l'inverse de l'objectif recherché (TR1 [R0] : *"optimize performance across all systems"*). La dénormalisation assumée dans les dimensions coûte en stockage mais pas en temps de requête.

D3.3. Les étoiles des processus métiers

Les diagrammes ci-dessous représentent le schéma en étoile de chaque processus métier. Les six étoiles sont regroupées en trois sous-ensembles fonctionnels : *Transactions*, *Abonnements & Fraudes*, *Sécurité & Conformité*. Chaque processus a son propre grain.

D3.3.1. Revenus (P1/FactTransaction)



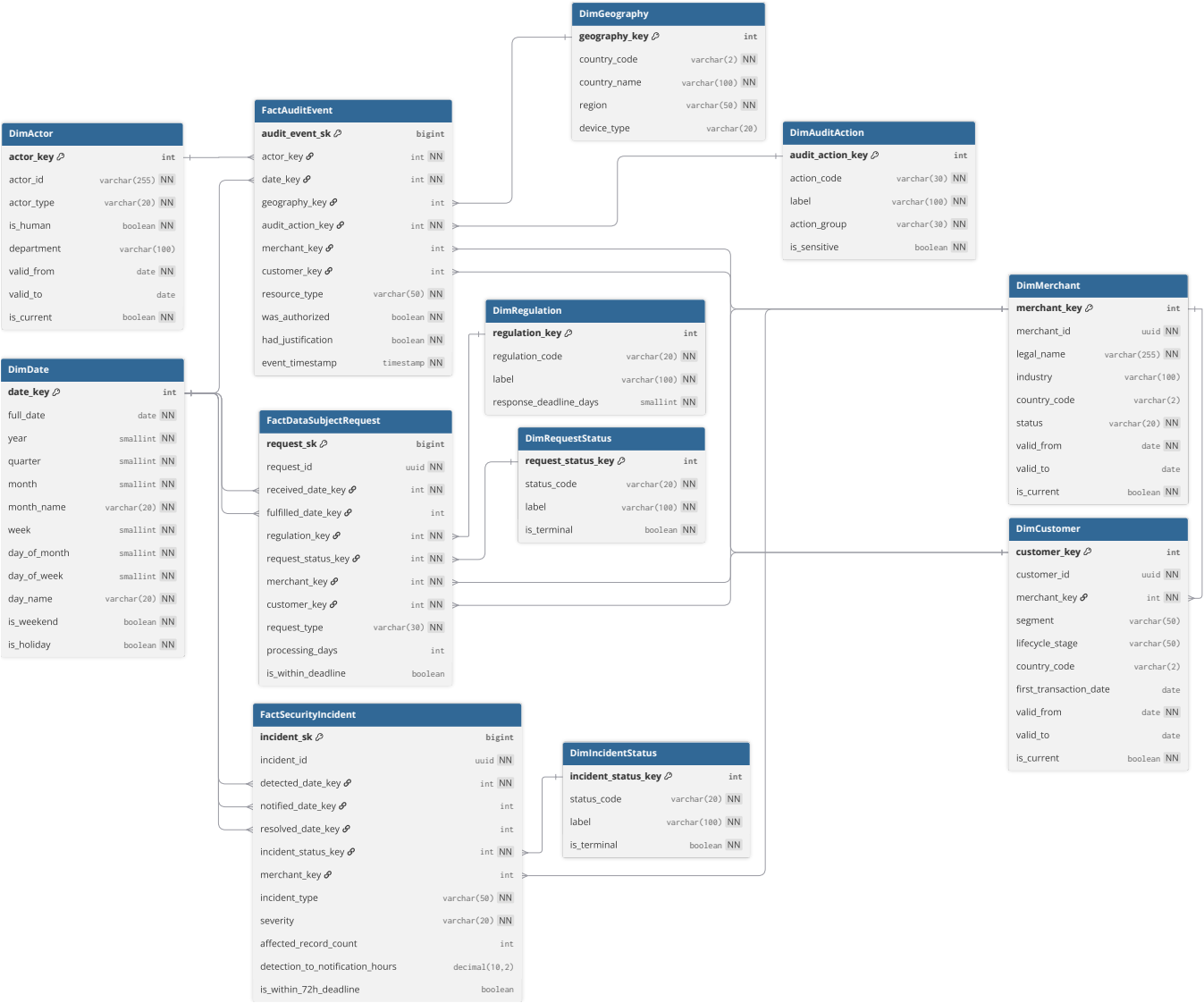
Grain : une transaction. Les propriétés `merchant_net_revenue_eur` et `stripe_revenue_eur` cohabitent sur la même ligne. Les deux audiences du reporting (marchands & Stripe) lisent la même table, seule la colonne agrégée change (voir note du DBML).

D3.3.2. Fraudes (P2/FactFraudEvent) & Abonnements (P3/FactSubscriptionSnapshot)



Deux grains différents dans la même étoile fonctionnelle : FactFraudEvent (un événement d'évaluation) et FactSubscriptionSnapshot (un instantané d'abonnement capturé périodiquement). Les tables ne partagent aucune ligne, seulement des dimensions conformées.

D3.3.3. Conformité & Sécurité (P4/FactAuditEvent, P5/FactDataSubjectRequest, P6/FactSecurityIncident)



FactDataSubjectRequest et FactSecurityIncident référencent DimDate trois fois chacune (réception/traitement, détection/notification/résolution). C'est la signature typique d'un instantané cumulatif *Kimball* : plusieurs jalons temporels sur une même ligne, mise à jour au fur et à mesure du processus, au lieu de créer une nouvelle ligne par événement.

D3.4. Stratégie d'agrégation

Principe constant : *le fait atomique reste au grain le plus fin, les agrégats sont des tables séparées, calculées par l'ETL, jamais l'inverse. C'est ce principe qui a motivé le changement de grain de `FactSubscriptionSnapshot` : les compteurs agrégés qui y vivaient directement ont été déplacés vers une table dédiée.

Table d'agrégat	Grain	Alimentée depuis	Sert
<code>AggRevenueDailyMerchant</code>	Jour × marchand	<code>FactTransaction</code>	Tableau de bord marchand (quasi temps réel)
<code>AggRevenueMonthlyMerchantCountry</code>	Mois × marchand × pays	<code>FactTransaction</code>	Reporting géographique
<code>AggFraudDailyMerchant</code>	Jour × marchand	<code>FactFraudEvent</code>	Alerting fraude
<code>AggSubscriptionDailyMerchant</code>	Jour × marchand × produit	<code>FactSubscriptionSnapshot</code>	Tableau de bord abonnements
<code>AggComplianceDailyMerchant</code>	Jour × marchand	<code>FactAuditEvent</code>	Reporting conformité automatisé
<code>AggDataSubjectRequestMonthly</code>	Mois × marchand × réglementation × type	<code>FactDataSubjectRequest</code>	Rapport RGPD/CCPA
<code>AggSecurityIncidentMonthly</code>	Mois × marchand × sévérité	<code>FactSecurityIncident</code>	Rapport sécurité

Dénormalisation différenciée entre fait atomique et agrégat : le fait atomique référence toujours une dimension par clé (`regulation_key` , par exemple) pour permettre n'importe quel angle d'analyse ; l'agrégat précalculé garde parfois l'attribut en clair (`regulation varchar` dans `AggDataSubjectRequestMonthly`) pour être interrogeable sans jointure - un agrégat n'a pas vocation à être recombiné avec d'autres dimensions, juste à répondre vite à une question déjà connue.

D3.5. Techniques d'optimisation des requêtes

Clés de substitution entières plutôt qu'UUID. Plus compactes, comparaisons plus rapides, et nécessaires de toute façon pour le versionnement SCD Type 2 (une même `merchant_id` OLTP correspond à plusieurs `merchant_key` au fil du temps).

DimDate précalculée. Évite les fonctions de date dans les clauses `WHERE` (`EXTRACT` , `DATE_TRUNC` ...), qui empêchent l'élagage de partitions sur la plupart des moteurs colonne. Permet aussi les comparaisons période sur période par simple jointure sur `date_key` .

Partitionnement par `date_key` . Systématique sur les tables de faits à fort volume (`FactTransaction` , `FactAuditEvent`), mensuel ou quotidien selon le volume réel ; l'élagage de partitions élimine la majorité des lignes avant même le scan.

Stockage orienté colonne. *Redshift*, *BigQuery*, *Snowflake* ou *ClickHouse* ; cohérent avec des requêtes qui agrègent peu de colonnes sur beaucoup de lignes, l'inverse du pattern OLTP.

Diffusion des dimensions (*broadcast join*). Les dimensions restent petites en volume (quelques milliers à millions de lignes, jamais à l'échelle des faits) - elles peuvent être répliquées sur chaque nœud de calcul plutôt que redistribuées, évitant un *shuffle* coûteux lors de la jointure avec une table de faits partitionnée.

Taux de change figé (`DimReferenceExchangeRate`). Au-delà de la comparabilité déjà justifiée (2.A.7), c'est aussi une optimisation : convertir au vol au taux du jour demanderait une jointure sur une dimension à cardinalité temporelle fine ; le taux figé par exercice réduit drastiquement le nombre de versions à joindre.

Sécurité au niveau des lignes (RLS), pas de filtrage applicatif. Un marchand consultant son reporting ne doit voir que ses propres lignes (`merchant_key`) ; imposé par une politique

RLS au niveau du moteur, jamais laissé à la charge de la requête applicative, pour qu'aucun bug côté client ne puisse exposer les données d'un autre marchand.

Index ciblés sur les clés de jointure et les colonnes de filtrage fréquent - `date_key` et `(merchant_key, date_key)` systématiques sur les faits, complétés par les clés métier (`transaction_id`, `subscription_id` ...) pour le dédoublonnage à l'ETL et le rapprochement avec l'OLTP.

D3.6. Couverture des exigences

Exigence [R0]	Couverture
BR2 - analyse de revenu, segmentation, conformité	Six étoiles couvrant chacun des besoins (D3.3)
TR1 - normalisation OLTP / performance OLAP	Dénormalisation assumée, justifiée section D3.2
TR3 - scalabilité, indexation, partitionnement	Partitionnement par <code>date_key</code> , index ciblés (D3.5)

Références

Ressources

- [R0] [Stripe Business Case](#)